

CSE4502 Operating Systems Lab 1B

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

February 26, 2026

1 Part A: File Size Monitor (Conditionals and Arithmetic)

1.1 Problem Statement

Develop a Bash script named `file_size.sh` that checks the size of a file and classifies it as small, medium, or large based on user-defined thresholds.

1.2 Logic

1. Take a filename as input and verify its existence.
2. Use `size=$(stat -c%s $filename)` to get the file size in bytes.
3. Compare the size against thresholds:
 - `< 1000` bytes -> Small file
 - `1000–10000` bytes -> Medium file
 - `> 10000` bytes -> Large file
4. Display the classification.

1.3 Example Output

```
$ ./file_size.sh data.log
File: data.log
Size: 15432 bytes
Category: Large file
```

2 Part B: Multi-Level Priority-Based Concurrent Logging

Concurrent threads generate log messages with one of k **priority levels**, denoted as $P_1, P_2, \dots, P_k$, where P_1 is the **highest priority** and P_k is the **lowest priority**.

The logging system must ensure that all messages of priority P_i are processed and printed **before** any message of priority P_j for all $i < j$, regardless of their arrival order within the critical section.

2.1 Input (Thread Actions)

The system consists of kn concurrent threads:

- n threads producing log messages with priority P_1 (highest priority).
- n threads producing log messages with priority P_2 .
- $\vdots$
- n threads producing log messages with priority P_k (lowest priority).

Each thread generates exactly one log message and attempts to insert it into a shared logging buffer.

2.2 Constraints

- $1 \leq n \leq 40$ | $2 \leq k \leq 10$.
- You shouldn't sort the output.
- Can you do this without a counter variable or a shared state to track how many threads are done?
- Protect the shared buffer (conceptually equivalent to `stdout` in the case of `printf()`).
- Log insertion and ordering must be thread-safe.
- Messages with higher priority must not be delayed by messages of lower priority that arrive earlier.
- All messages of priority P_i must be printed before any message of priority P_j for all $i < j$.

2.3 Expected Output (Buffer Dump)

After all threads have completed execution, the contents of the logging buffer must satisfy the following ordering:

```
[P1] <Message 1>
[P1] <Message 2>
...
[P1] <Message n1>
[P2] <Message 1>
...
[P2] <Message n2>
...
[Pk] <Message 1>
...
[Pk] <Message nk>
```

The logging buffer is a formal nomenclature. Utilizing basic console prints for the messages is sufficient, as long as each print operation is atomic.